Janus API Gateway — Developer User Guide

> This guide assumes Janus is deployed behind Nginx with TLS enabled. Replace hostnames, ports, and sample keys with your own values.

1) Getting Started

Prerequisites

- Docker (Engine) installed
- Docker Compose installed (Compose v2 recommended)

Verify:

```
docker --version
docker compose version
```

One-command deployment

From the repository root (where `docker-compose.yml` lives):

```
docker compose up -d --build
```

Typical containers you should see:

- `janus-api-gateway` (Spring Boot)
- `janus-postgres` (PostgreSQL)
- `janus-redis` (Redis)
- `janus-nginx` (Nginx)

Check status:

```
docker compose ps
```

Generating SSL certificates (dev / local)

If you're using a local Nginx TLS termination, generate a self-signed cert:

```
mkdir -p certs
openssl req -x509 -newkey rsa:4096 -sha256 -days 365 -nodes \
  -keyout certs/janus.key \
  -out certs/janus.crt \
  -subj "/CN=localhost"
```

Update your Nginx config to point to `certs/janus.crt` and `certs/janus.key` (path depends on your compose volume mounts).

For production, use a trusted CA (e.g., Let's Encrypt) and rotate certificates per your security policy.

Verifying health endpoints

Janus exposes two types of health endpoints:

1. **Gateway health** (Janus-specific)
2. **Spring Boot Actuator** (platform-level)

Assuming Nginx exposes HTTPS on `https://localhost`:

```
curl -sk https://localhost/api/health | jq
curl -sk https://localhost/actuator/health | jq
```

Expected `GET /api/health` example response:

```
{
  "status": "UP",
  "service": "janus-api-gateway",
  "redis": {
    "status": "UP",
    "latency_ms": 2
  },
  "postgres": {
    "status": "UP"
  },
  "time": "2026-05-11T14:05:12Z"
}
```

2) API Key Management

Janus uses API keys to identify a consumer and enforce plan-based limits. API keys are created via the management API.

Plans

- **free**: 10 req/s
- **pro**: 100 req/s
- **enterprise**: 1000 req/s
- **payg** (pay-as-you-go): $0.01 per request, no fixed RPS ceiling (still subject to fair-use / safety rails at the edge)

Create an API key (curl)

Endpoint (example):

- `POST /api/keys`

Payload fields (typical):

- `name`: label for humans
- `plan`: `free | pro | enterprise | payg`
- `owner`: optional identifier/email/team

Free plan

```
curl -sk -X POST https://localhost/api/keys \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "local-dev-free",
    "plan": "free",
    "owner": "dev-team"
  }' | jq
```

Example response:

```
{
  "id": "key_01J2FQJ3FQ2A9JY7Y7Z4YH1M1Q",
  "plan": "free",
  "name": "local-dev-free",
  "created_at": "2026-05-11T14:09:44Z",
  "masked_key": "janus_live_8f3a…c19d",
```

```
  "raw_key": "janus_live_8f3a9e7b2c0c4d1b9a54a0f1c19d"
}
```

> ⚠ **Important:** The `raw_key` is returned **once** at creation time and is **never stored** in plaintext. If you lose it, you must rotate (create a new key and revoke the old one).

Pro plan

```
curl -sk -X POST https://localhost/api/keys \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "billing-service-pro",
    "plan": "pro",
    "owner": "platform"
  }' | jq
```

Enterprise plan

```
curl -sk -X POST https://localhost/api/keys \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "partner-x-enterprise",
    "plan": "enterprise",
    "owner": "bd"
  }' | jq
```

Pay-as-you-go plan

```
curl -sk -X POST https://localhost/api/keys \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "experiment-payg",
    "plan": "payg",
    "owner": "growth"
  }' | jq
```

Masked key format

Janus returns a masked key for display/logging:

- Example: `janus_live_8f3a…c19d`
- Pattern: prefix + first 4–8 chars + ellipsis + last 4 chars

Use `masked_key` in UIs, logs, dashboards, and support tickets. Use `raw_key` only in secure secrets storage.

---

3) Authentication

Janus supports two authentication modes for downstream APIs:

1. **Direct API key authentication** using the `X-API-Key` header
2. **JWT authentication** by exchanging an API key for access/refresh tokens

A) Direct API key auth (X-API-Key)

Send your key on every request:

```
curl -sk https://localhost/api/proxy/v1/customers \
  -H 'X-API-Key: janus_live_8f3a9e7b2c0c4d1b9a54a0f1c19d' \
  -H 'X-Request-ID: req_9f2c6b2a2f2b4b1a'
```

Example response:

```
{
  "data": [
    {"id": "cus_1001", "name": "Acme"},
    {"id": "cus_1002", "name": "Globex"}
  ],
  "request_id": "req_9f2c6b2a2f2b4b1a"
}
```

B) Exchange API key for JWT

Use JWTs for production applications to avoid sending API keys from client-side apps.

Endpoint (example):

- `POST /api/auth/token`

```
curl -sk -X POST https://localhost/api/auth/token \
  -H 'Content-Type: application/json' \
```

```
  -H 'X-Request-ID: req_4d1c8c1a0e3b4e9a' \
  -d '{
    "api_key": "janus_live_8f3a9e7b2c0c4d1b9a54a0f1c19d"
  }' | jq
```

Example response:

```
{
  "token_type": "Bearer",
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJrZXl
fMDFK...",
  "expires_in": 3600,
  "refresh_token": "rft_01J2FR1F6GJ2Q5P9X9C1D2E3F4",
  "refresh_expires_in": 2592000
}
```

- Access token TTL: **1 hour (3600s)**
- Refresh token TTL: **30 days (2592000s)**

C) Call APIs using Authorization: Bearer

```
curl -sk https://localhost/api/proxy/v1/customers \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdW
IiOiJrZXlfMDFK...'
```

D) Refresh an expired access token

Endpoint (example):

- `POST /api/auth/refresh`

```
curl -sk -X POST https://localhost/api/auth/refresh \
  -H 'Content-Type: application/json' \
  -d '{
    "refresh_token": "rft_01J2FR1F6GJ2Q5P9X9C1D2E3F4"
  }' | jq
```

Example response:

```
{
  "token_type": "Bearer",
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJrZXl
```

```
fMDFK...",
  "expires_in": 3600
}
```

Common auth errors

- Missing key/header → `400`
- Invalid API key → `401`
- Expired/invalid JWT → `401`

---

4) Rate Limiting

Janus enforces limits at multiple layers:

1. **Nginx edge protection** (pre-gateway)
2. **Gateway rate limiting** (Redis sliding window, millisecond precision)
3. **Optional downstream protections** (your services)

Plan tiers

- **free**: 10 requests/second
- **pro**: 100 requests/second
- **enterprise**: 1000 requests/second
- **payg**: $0.01/request (usage-based). Requests are metered and billed; operational guardrails still apply.

How sliding window limiting works (Redis Sorted Sets)

Janus implements sliding window rate limiting per API key (and optionally per route):

- Each request inserts an entry into a Redis **sorted set** (ZSET)
- **Score** = request timestamp in **milliseconds**
- Janus removes entries older than the window (e.g., last 1000ms)
- The request is allowed if the count of entries in the current window is ≤ the plan's threshold

Why ZSET:

- Efficient range deletes by score
- Accurate to milliseconds (reduces "boundary" loopholes vs fixed windows)

Nginx edge protection (3-layer defense)

At the edge, Nginx can reduce load before requests reach the gateway:

1. **Connection limiting**: caps concurrent connections per client IP
2. **Static rate limiting**: basic `limit_req` per IP (coarse but fast)
3. **Micro-cache**: caches identical GET responses briefly (e.g., 50–200ms), smoothing bursts

This protects Janus/Redis/PostgreSQL from spikes and helps keep latency stable.

Example: Burst 15 requests with a free key

A free key allows ~10 req/s. With a burst of 15 within 1 second, you should observe:

- First **9** succeed (200)
- Requests **10+** get **429**

Run:

```
for i in $(seq 1 15); do
  code=$(curl -sk -o /dev/null -w "%{http_code}" \
    https://localhost/api/proxy/v1/ping \
    -H 'X-API-Key: janus_live_8f3a9e7b2c0c4d1b9a54a0f1c19d');
  echo "$i -> $code";
done
```

Expected output (example):

```
1 -> 200
2 -> 200
3 -> 200
4 -> 200
5 -> 200
6 -> 200
7 -> 200
8 -> 200
9 -> 200
10 -> 429
11 -> 429
12 -> 429
13 -> 429
14 -> 429
15 -> 429
```

Example 429 response body:

```
{
  "error": "rate_limit_exceeded",
  "message": "Plan 'free' allows 10 req/s. Please retry later.",
```

```
  "retry_after_ms": 120,
  "request_id": "req_7c2e9a1b3f9d4c10"
}
```

Handling 429 responses (exponential backoff in JavaScript)

```
async function fetchWithBackoff(url, options = {}, maxRetries = 5) {
  let attempt = 0;
  let delay = 100; // ms

  while (true) {
    const res = await fetch(url, options);

    if (res.status !== 429) return res;

    attempt += 1;
    if (attempt > maxRetries) throw new Error("Too many retries (42
9)");

    const retryAfterMs = Number(res.headers.get("Retry-After-Ms")) ||
0;
    const wait = Math.max(delay, retryAfterMs);

    await new Promise(r => setTimeout(r, wait));
    delay *= 2;
  }
}
```

Notes:

- Prefer respecting server-provided retry hints (`retry_after_ms` or `Retry-After` headers).
- Add jitter in high-concurrency clients to avoid synchronized retries.

---

5) Dynamic Rules (Hot-Reload)

Janus uses Drools to apply dynamic routing, validation, enrichment, and policy decisions.

How it works

- Rules are stored in **PostgreSQL** (for example in a `rules` table)

- On create/update, Janus loads the rule definition and compiles it into a Drools knowledge base at runtime
- Rules take effect immediately—no restart required

Create a new rule via API

Endpoint (example):

- `POST /api/rules`

Rule example: block a specific header value on a route.

```
curl -sk -X POST https://localhost/api/rules \
  -H 'Content-Type: application/json' \
  -H 'X-API-Key: janus_live_8f3a9e7b2c0c4d1b9a54a0f1c19d' \
  -d '{
    "name": "BlockBadClient",
    "enabled": true,
    "salience": 10,
    "when": {
      "route": "/api/proxy/v1/customers",
      "header": "X-Client",
      "equals": "bad-bot"
    },
    "then": {
      "action": "REJECT",
      "status": 401,
      "message": "Client blocked by policy"
    }
  }' | jq
```

Example response:

```
{
  "id": "rule_01J2FRB3K2B0R9FJ9V7W1C2D3E",
  "name": "BlockBadClient",
  "enabled": true,
  "compiled": true,
  "updated_at": "2026-05-11T14:22:10Z"
}
```

Toggle rules on/off

Endpoint examples:

- `PATCH /api/rules/{id}`

Disable:

```
curl -sk -X PATCH https://localhost/api/rules/rule_01J2FRB3K2B0R9FJ9V7W
1C2D3E \
  -H 'Content-Type: application/json' \
  -H 'X-API-Key: janus_live_8f3a9e7b2c0c4d1b9a54a0f1c19d' \
  -d '{"enabled": false}' | jq
```

Enable:

```
curl -sk -X PATCH https://localhost/api/rules/rule_01J2FRB3K2B0R9FJ9V7W
1C2D3E \
  -H 'Content-Type: application/json' \
  -H 'X-API-Key: janus_live_8f3a9e7b2c0c4d1b9a54a0f1c19d' \
  -d '{"enabled": true}' | jq
```

Reload all rules from the database

Endpoint (example):

- `POST /api/rules/reload`

```
curl -sk -X POST https://localhost/api/rules/reload \
  -H 'X-API-Key: janus_live_8f3a9e7b2c0c4d1b9a54a0f1c19d' | jq
```

Example response:

```
{
  "reloaded": true,
  "rule_count": 27,
  "compiled": 27,
  "time_ms": 83
}
```

---

6) Monitoring & Audit

Gateway health: /api/health

Use this for operational checks and dependency visibility.

```
curl -sk https://localhost/api/health | jq
```

This endpoint should include Redis connectivity status (and often PostgreSQL status).

Actuator endpoints

Common endpoints (enable/secure per environment):

- `GET /actuator/health`
- `GET /actuator/metrics`

```
curl -sk https://localhost/actuator/health | jq
curl -sk https://localhost/actuator/metrics | jq
```

Structured logging with trace IDs

Send a request ID to correlate client errors ↔ gateway logs ↔ audit events:

- Request header: `X-Request-ID: <string>`

Example:

```
curl -sk https://localhost/api/proxy/v1/ping \
  -H 'X-API-Key: janus_live_8f3a9e7b2c0c4d1b9a54a0f1c19d' \
  -H 'X-Request-ID: req_0b7a2a9f1b3c4d5e'
```

Janus should:

- Log the `request_id`
- Include it in error payloads when possible

Audit events (PostgreSQL)

Janus persists audit events to PostgreSQL (example table: `event_logs`). Typical fields:

- `id`
- `created_at`
- `request_id`
- `api_key_id` (or consumer id)
- `route`
- `action` (ALLOW / DENY / RULE_APPLIED / KEY_CREATED / TOKEN_ISSUED)
- `status_code`

- `metadata` (JSON)

Example SQL queries

1) Latest denied requests

```
SELECT id, created_at, request_id, api_key_id, route, status_code, acti
on
FROM event_logs
WHERE action = 'DENY'
ORDER BY created_at DESC
LIMIT 50;
```

2) Audit trail for a request ID

```
SELECT created_at, action, status_code, route, metadata
FROM event_logs
WHERE request_id = 'req_0b7a2a9f1b3c4d5e'
ORDER BY created_at ASC;
```

3) Top rate-limited keys (last 24h)

```
SELECT api_key_id, COUNT(*) AS limited_count
FROM event_logs
WHERE action = 'DENY'
  AND status_code = 429
  AND created_at >= NOW() - INTERVAL '24 hours'
GROUP BY api_key_id
ORDER BY limited_count DESC
LIMIT 20;
```

---

7) Error Codes Reference

Status	Meaning	Common causes	What to do
200	Success	—	Continue
400	Bad Request	Missing headers, invalid JSON	Fix request format
401	Unauthorized	Invalid API key, expired/invalid JWT	Verify key/token; refresh token
429	Rate limit exceeded	Plan threshold exceeded	Backoff; respect retry hints

Status	Meaning	Common causes	What to do
500	Internal Server Error	Unhandled exception, misconfiguration	Check logs with request ID
503	Service Unavailable	Nginx edge returning cached/stale response or upstream unavailable	Retry; check upstream health

429 responses should include a retry hint when available:

- Body field: `retry_after_ms`
- Or a response header such as `Retry-After` / `Retry-After-Ms`

---

8) Security Best Practices

- Store API keys in **environment variables** or a secrets manager (Vault, AWS Secrets Manager, etc.).
- Rotate keys periodically (and immediately after suspected exposure).
- Prefer **JWT** for production applications, especially for browser/mobile clients.
- Never commit `.env` files or secrets into source control.
- Send `X-Request-ID` on every request for supportability and debugging.
- Restrict management endpoints (API key creation, rule updates) to trusted networks/roles.
- Enable TLS everywhere (edge → gateway → downstream where feasible).

---

9) Plan Comparison

Plan	Price	Rate limit	APIs	API keys	Support
Free	$0	10 req/s	1	1	Community
Pro	$49/mo	100 req/s	10	10	Email
Enterprise	Custom	1000 req/s	Unlimited	Unlimited	Priority
Pay-as-you-go	$0.01/req	Usage-based	Unlimited	1	Community

Quick integration checklist

- [ ] Deploy with Docker Compose
- [ ] Verify `/api/health` and `/actuator/health`
- [ ] Create API key; store `raw_key` securely
- [ ] Integrate with `X-API-Key` or JWT exchange
- [ ] Implement 429 backoff + retries

- [ ] Use `X-Request-ID` and log correlation
- [ ] Review and apply security best practices